



APRESENTAÇÃO

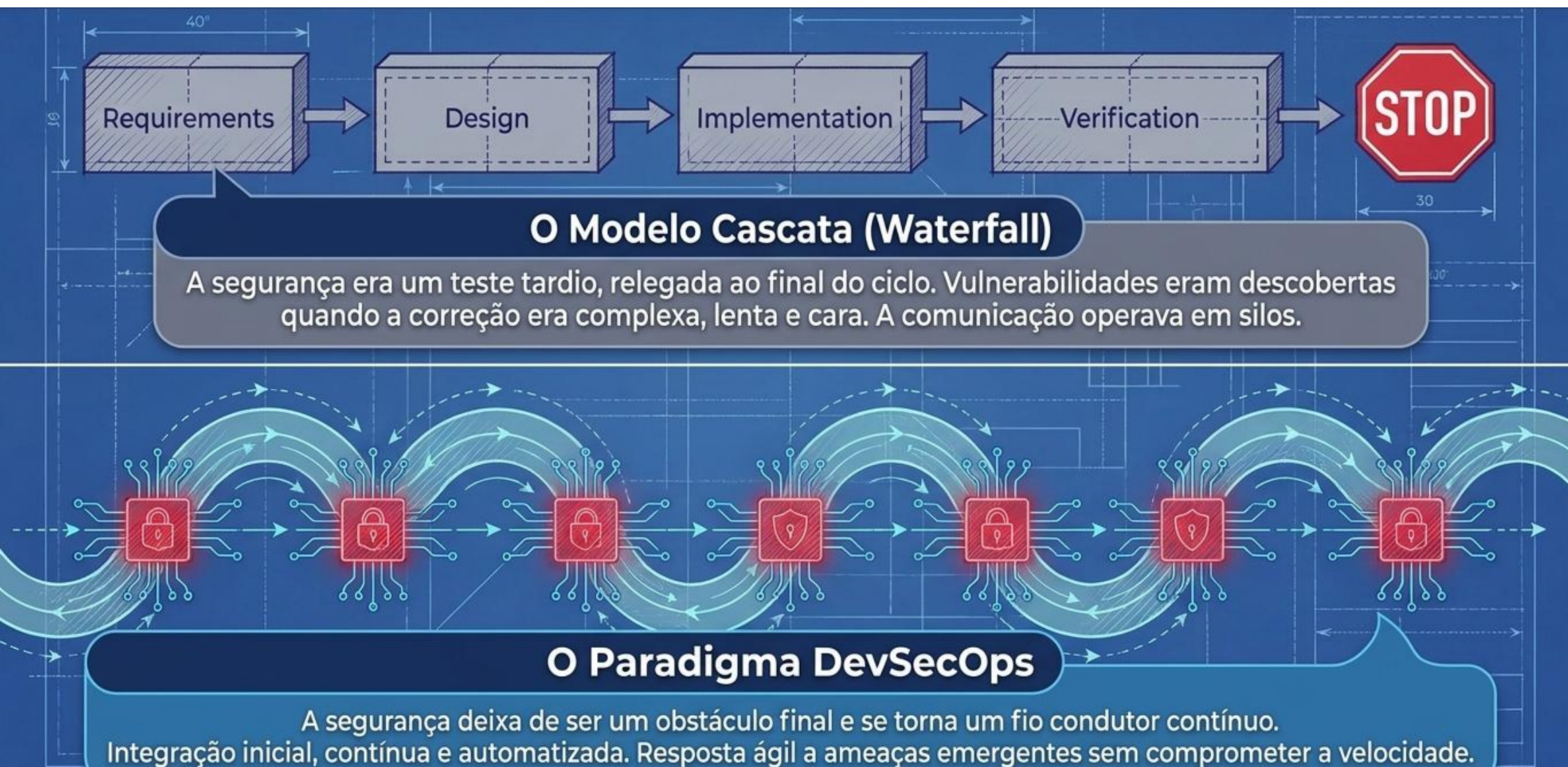
Segurança da Informação

Software seguro. CI-CD. DevSecOps

Prof. MSc Luis Gonzaga luis.gonzaga@pucpr.br



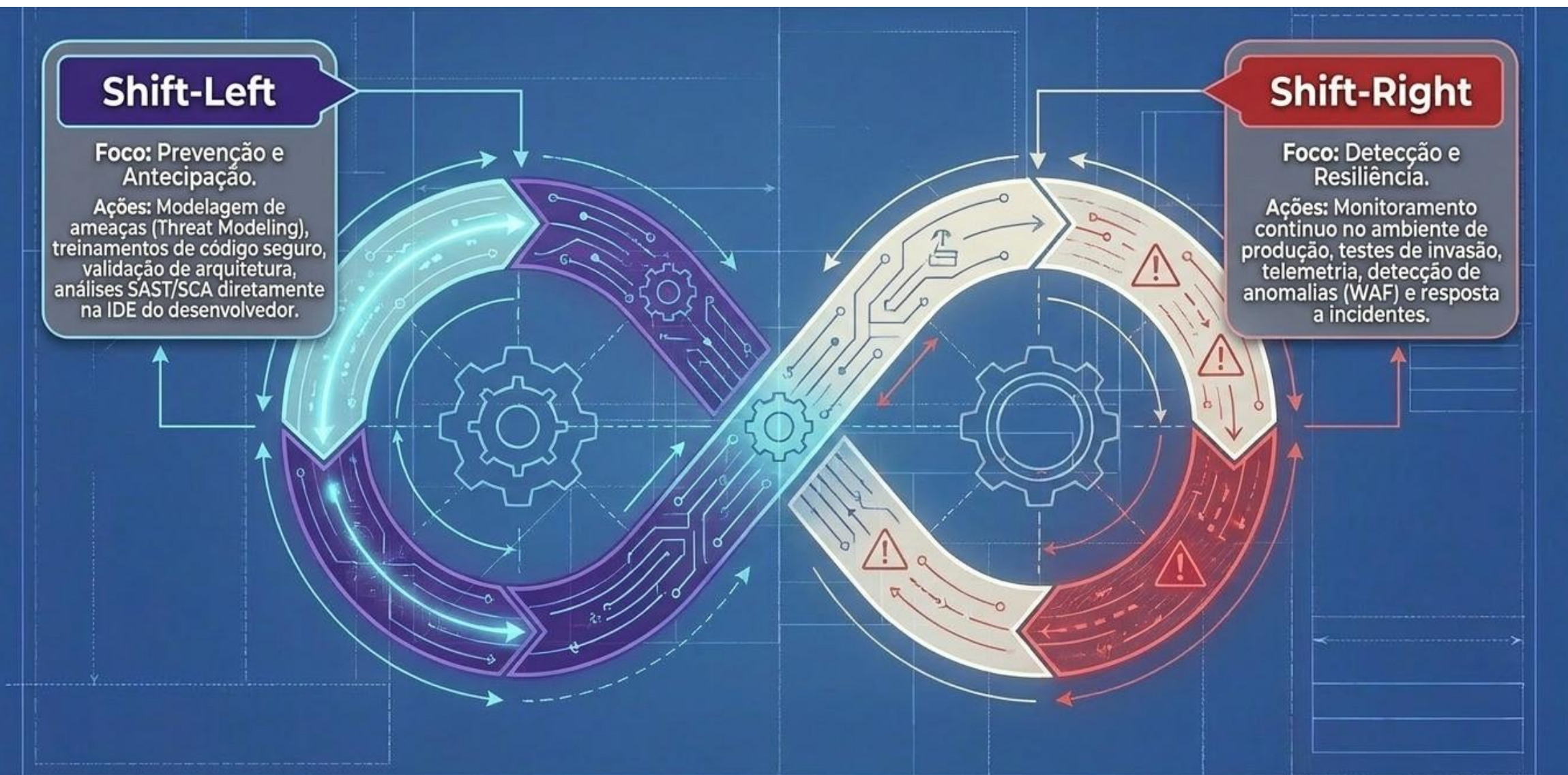
Da cascata ao DevSecOps



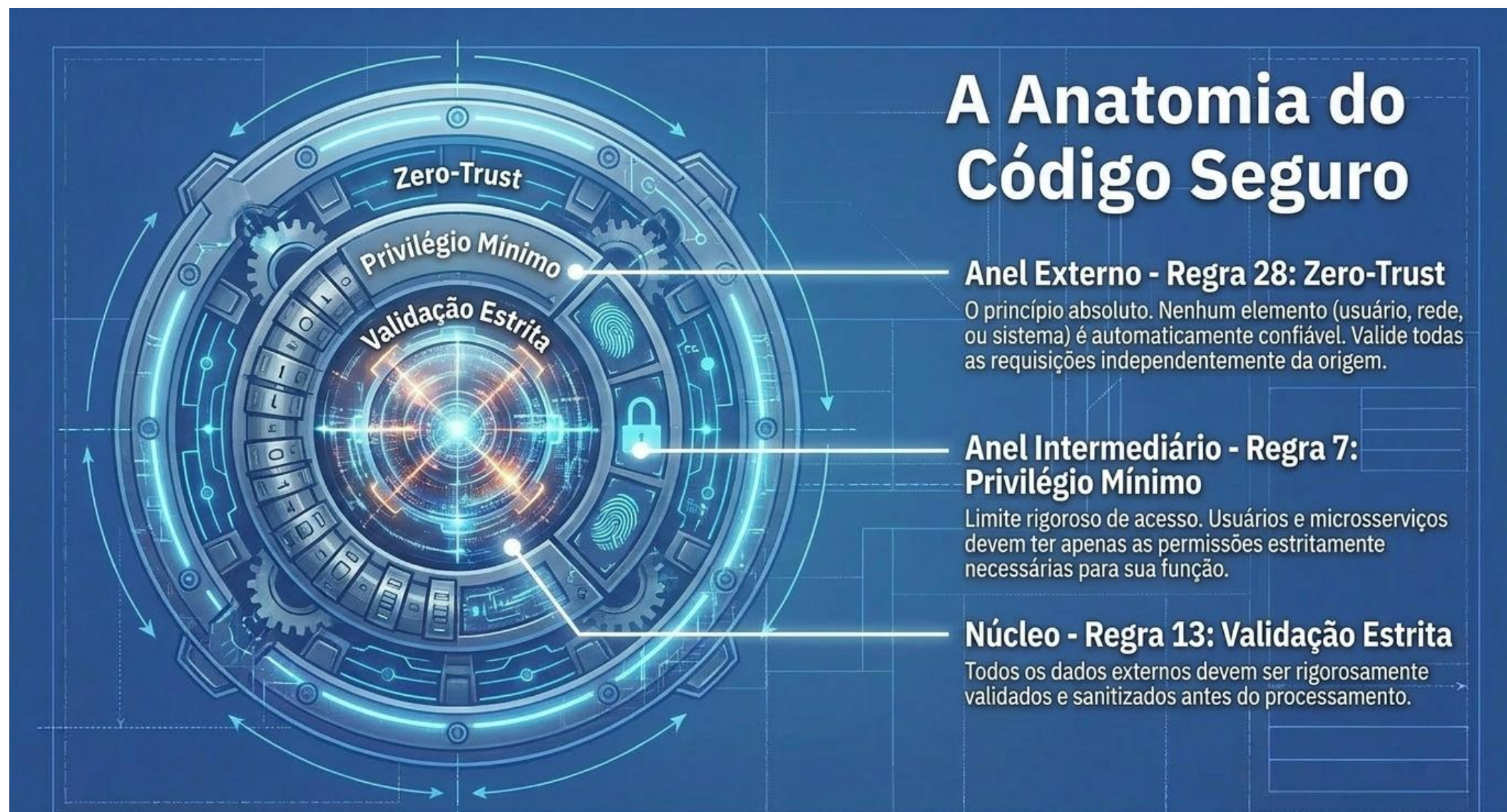
A cultura DevSecOps: responsabilidade compartilhada



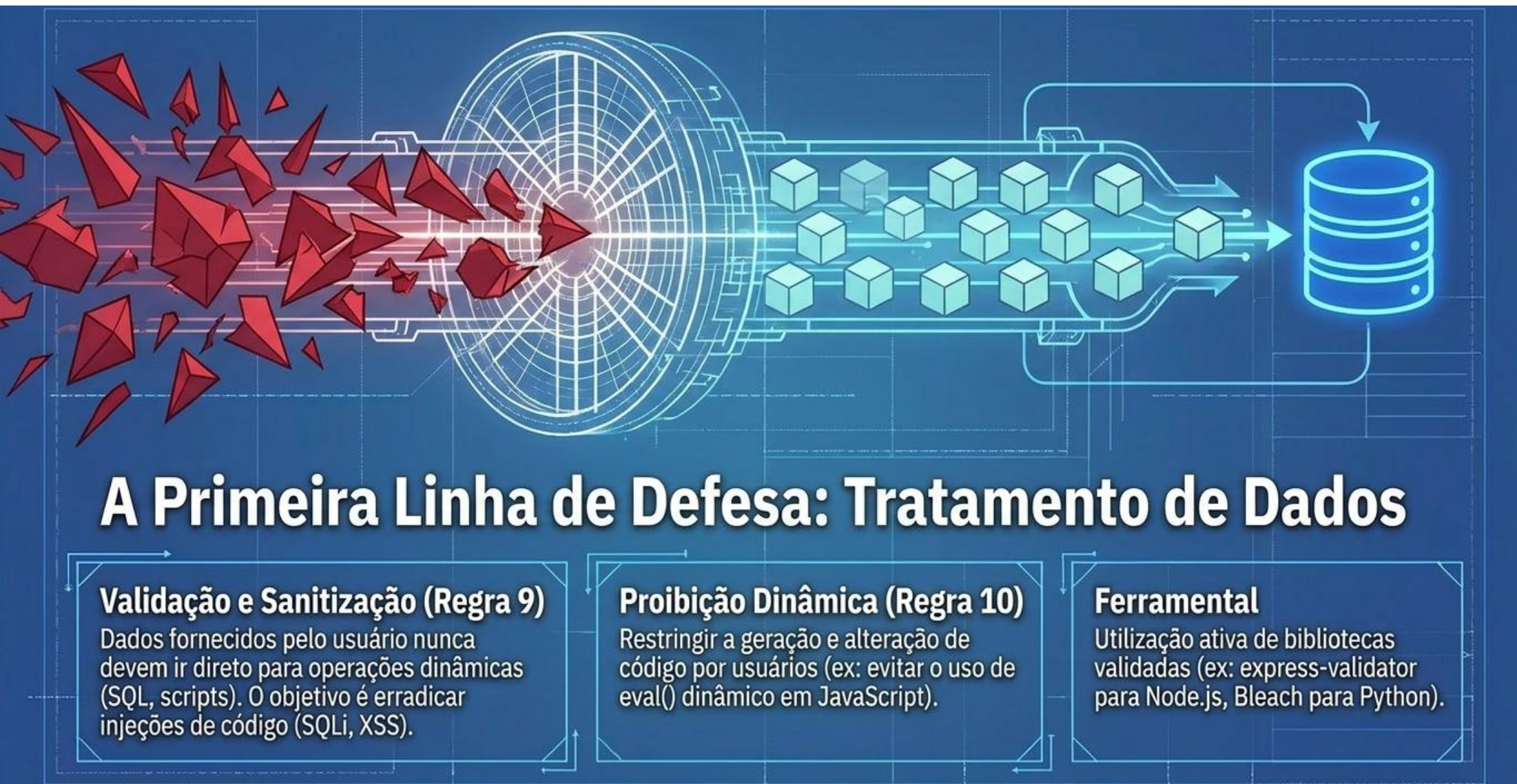
O loop infinito: shift-left e shift-right



Código seguro: o que realmente é isso?



Onde tudo começa...



Blindando os recursos

Segredos e Estado

Gestão de Segredos (Regra 22)

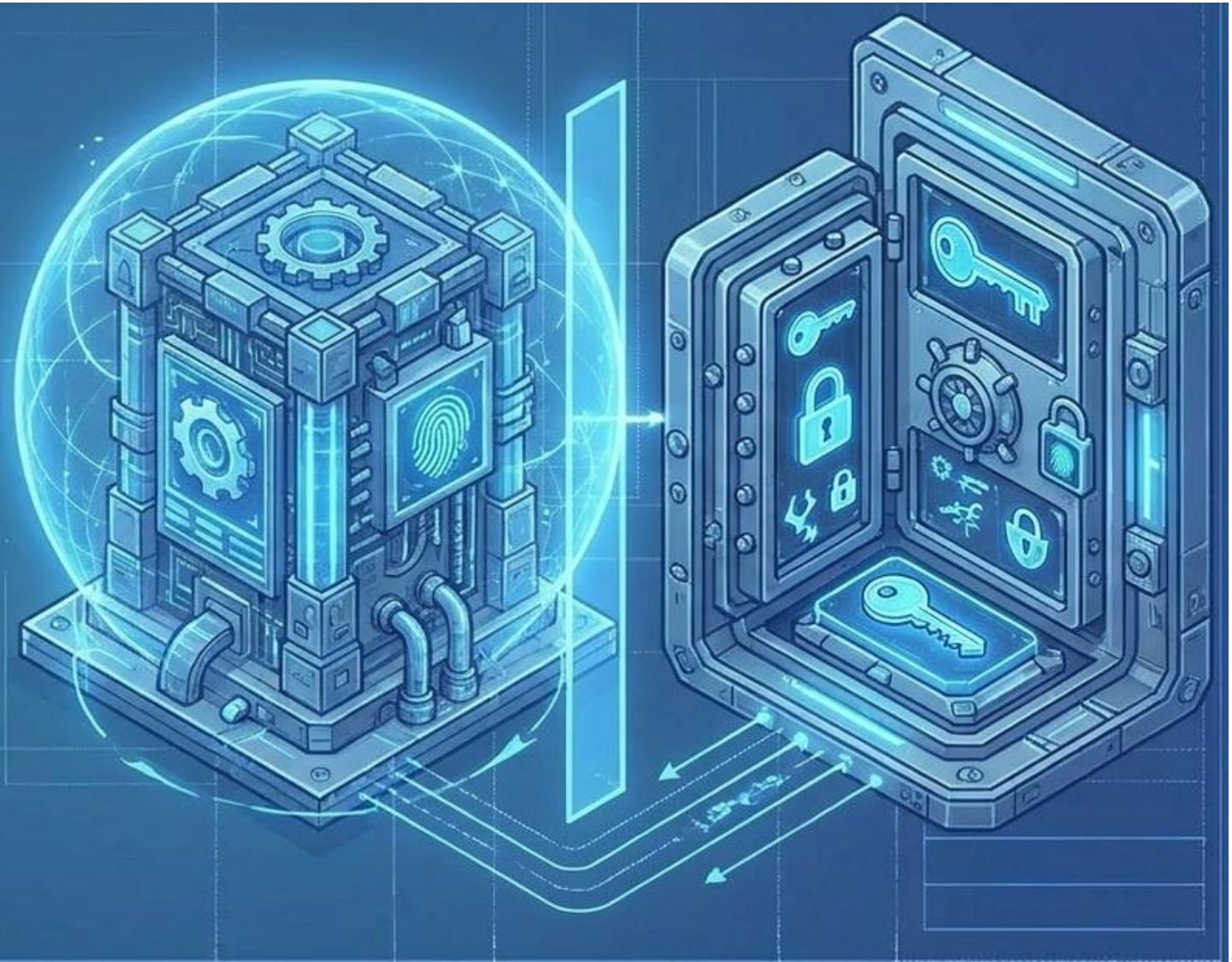
Nunca armazenar chaves de criptografia, tokens ou credenciais no código-fonte. Uso obrigatório de cofres de segredos (HashiCorp Vault, AWS Secrets Manager).

Variáveis e Estado (Regras 5 e 6)

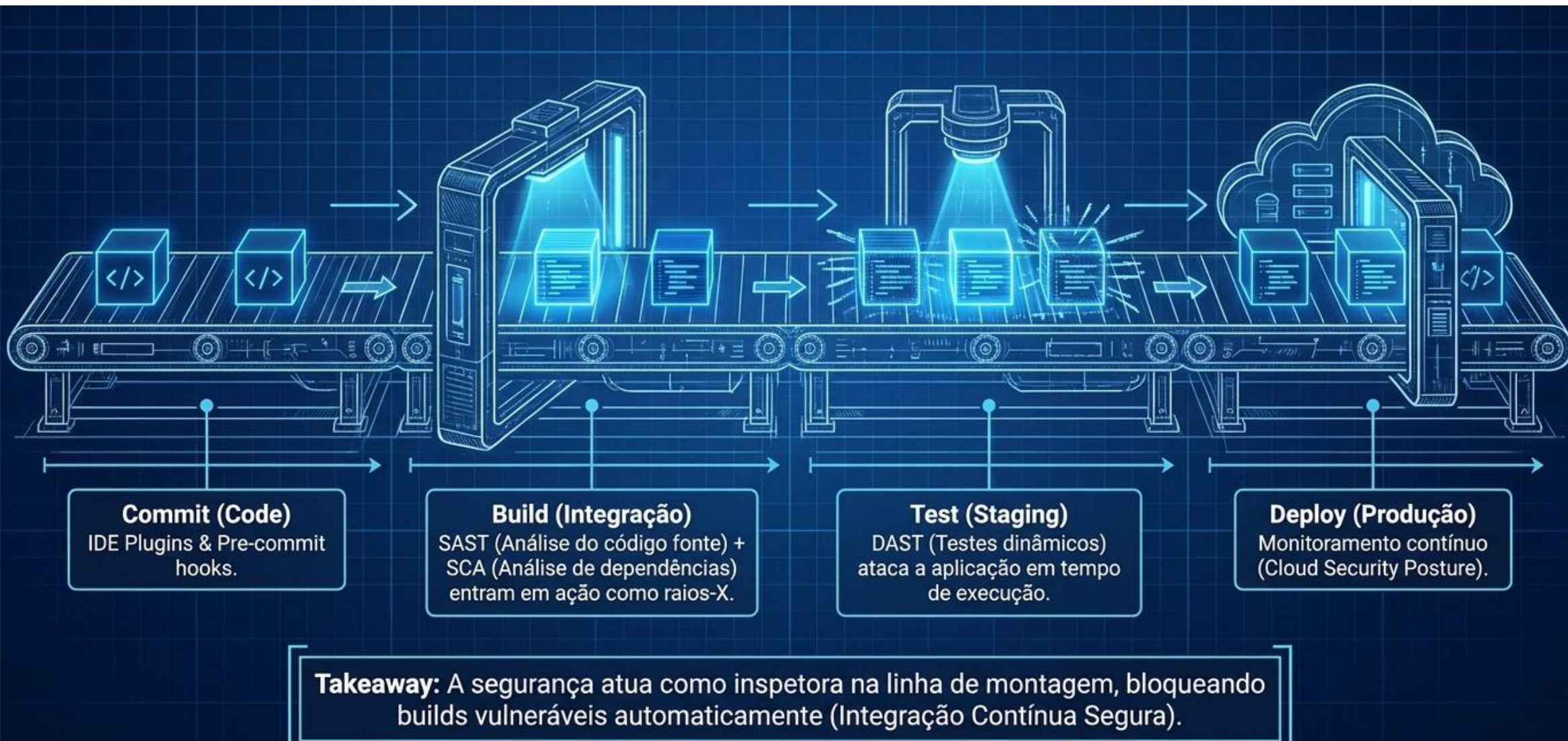
Proteger variáveis compartilhadas para impedir acessos concorrentes inseguros. Instanciar variáveis explicitamente para evitar comportamentos inesperados do sistema.

Imutabilidade (Regra 25)

Preferência por objetos imutáveis (ex: `Object.freeze()` em JS) para impedir alterações maliciosas em tempo de execução.



A esteira da automação: CI/CD



A matriz dos scanners: SAST x DAST x SCA

Fase	O que analisa	Velocidade	Falsos Positivos
SAST (Static)	Código Fonte (White-box) - Foco em falhas de lógica (SQLi, XSS)	Rápida	Altos (sem contexto de execução)
SCA (Composition)	Bibliotecas e Dependências Open-Source - Foco em CVEs conhecidos	Muito Rápida	Moderados
DAST (Dynamic)	Aplicação em execução (Black-box) - Foco em falhas de runtime	Lenta	Baixos (prova real de exploração)



SAST:

O que é:

Static Application Security Testing.

Analisa o código de dentro para fora, antes mesmo da compilação.



Vantagens e Ferramentas:

Altamente escalável;
garante a aderência às
políticas de codificação
segura (ex: regras do
Ministério da Saúde).

Ferramentas Comuns:
SonarQube, GitHub
CodeQL.

O Desafio:

Lidar com a alta taxa de falsos positivos e treinar a equipe para corrigir as falhas nativamente.



SCA: protegendo a “cadeia de suprimentos”

O que é:

Software Composition Analysis.
Mais de 70% das aplicações modernas são compostas por código de terceiros (open-source). A varredura de dependências cruza arquivos de manifesto (lockfiles) com bancos de dados de vulnerabilidades (NVD, CVEs).



Vantagens e Ferramentas:

Ferramentas Comuns:
Snyk, Dependabot, AquilaX.

O Desafio:

O Caso Log4j: Demonstrou como uma dependência transitiva profunda pode comprometer sistemas globais.



DAST: O “teste de colisão” dinâmico

O que é:

Dynamic Application Security Testing.

Testa a aplicação em sua forma final e em execução, replicando os passos de um atacante externo (fuzzing, injeções em formulários).



Vantagens e Ferramentas:

Valida como os componentes integrados se comportam juntos, revelando falhas que o SAST jamais veria.

Ferramentas Comuns:
OWASP ZAP, Burp Suite.

O Desafio:

Integração: Mais difícil de automatizar perfeitamente no CI/CD devido ao tempo de execução, frequentemente rodado em ambientes de homologação.



O fator humano: por que não basta “automatizar”?



Scanners (SAST/DAST/SCA) são excepcionais para encontrar vulnerabilidades conhecidas e erros de sintaxe (o básico bem feito).



Porém, scanners não compreendem o contexto do negócio nem falhas de lógica complexas (ex: burlar um fluxo de aprovação financeira). É aqui que entra o Pentest (Teste de Intrusão): profissionais de segurança simulando ataques criativos do mundo real para quebrar as defesas.



A MATRIZ DE PENTEST (ou as caixas de auditoria/teste)



Caixa Branca (White Box)

O auditor possui conhecimento total (código-fonte, topologia, credenciais). Simula ataques de ameaças internas (insiders).



Caixa Cinza (Gray Box)

Conhecimento parcial. Acelera os testes fornecendo o contexto que um atacante eventualmente descobriria.



Caixa Preta (Black Box)

Zero informação prévia (apenas o domínio/IP). Simula as condições exatas de um ataque externo cego.



Pentest na prática: a visão de quem ataca

Varredura de vulnerabilidades utilizando o Acunetix Web Vulnerability Scanner em duas aplicações web distintas (FATEC, 2015).

Cenário A (GLPI - Desenvolvimento Sem Foco em Segurança)

acunetix threat level

Level 3: High

Alerts distribution

Total alerts found

161

- High
- Medium
- Low
- Informational

1

71

6

83

161 vulnerabilidades encontradas (incluindo ameaças de nível Alto). Falta de regras rigorosas durante a engenharia.

Cenário B (Website Comercial - Foco em Segurança)

acunetix threat level

Apenas 2 vulnerabilidades (nível Baixo). A aplicação foi arquitetada desde o início prevendo exposição à internet.

Takeaway: O processo de desenvolvimento dita a superfície de ataque final.



SecDevOps: o equilíbrio, afinal?

O maior desafio da adoção do DevSecOps não é apenas técnico, é estratégico.

Se a segurança for muito rigorosa e manual, a agilidade do DevOps morre.

Se a velocidade for priorizada sem controle, o risco cibernético dispara.



A Solução: O ponto de equilíbrio (o fulcro) exige ferramentas de automação centradas no desenvolvedor (testes contínuos) e uma mudança cultural onde velocidade e segurança não sejam inimigas.



Concluindo: A Engenharia do Software Seguro

A segurança da informação não é um departamento.
Não é uma etapa final de testes.
E não é um selo de aprovação.

A segurança é um princípio arquitetural fundamental.

Ao adotar a cultura DevSecOps, escrever código estruturalmente robusto (Zero-Trust), automatizar a esteira (CI/CD) e auditar criticamente, nós paramos de lutar contra os atacantes no final da linha — nós construímos fortalezas imunes por design.





Obrigado!

Luis Gonzaga

luis.gonzaga@pucpr.br

